

NELSON MANDELA METROPOLITAN UNIVERSITY
SUMMERSTRAND CAMPUS (SOUTH)

SEMESTER TEST QUESTION PAPER

15 MAY 2015

SUBJECT: Advanced Programming 3.1

MODULE CODE: WRAP301

TIME : 3 hours

MARKS: 75

EXAMINERS:

MODERATORS:

1. Dr. D. Vogts

1. Dr. K. Naudé

SPECIAL INSTRUCTIONS

1. Answer **ALL** questions.
 2. This question paper consists of 2 sections:
 - Section A: 20 marks **written questions**.
 - Section B: 55 marks Java **practical questions**.
 3. Section A must be completed and handed in before Section B may be started.
 4. No permitted literature for Section A.
 5. Permitted literature for Section B:
 - JDK online documentation.
 - Bound lecture notes.
 - Online lecture notes.
 - Prescribed text book.
 - Internet Access. **Any file sharing or messaging sites are strictly forbidden!** Internet usage will be monitored during the test and any inappropriate access will result in disciplinary action being taken. If unsure about a site you would like to access, ask!
 6. Your solutions for the questions in this paper **must** be your own original work.
 7. **You are not printing your solutions, so you must save a copy of your code on the T: drive in the sub-folder allocated to you! Remember to include your name and student number in your source code files! No files = no marks!**
-

Section A: Written/Theory Questions**[20]**

*Note: You may **not** use your lecture notes, text book or JavaDocs when answering these questions!*

Note: don't forget to write your student number and name on the answer book!

Once completed, hand in your answers for Section A to receive Section B.

-
- | | |
|---|-----|
| 1) Briefly describe how the Model-View-Controller architecture works. | (5) |
| 2) JavaFX makes use of Stages, Scenes and Nodes. Briefly discuss each in turn. | (6) |
| 3) What is <i>Reflection</i> ? What does it allow you to do? | (3) |
| 4.1) What is <i>regression testing</i> and why is it important? | (2) |
| 4.2) When creating unit tests for a class: | |
| 4.2.1) It is not necessary to test each and every method. <i>Which</i> methods should tests be written for and <i>why</i> ? | (2) |
| 4.2.2) <i>How</i> should a method be tested? | (2) |

Section B: Programming Questions**[55]**

Use the provided username and password to log on.

Note: You may use your lecture notes, internet (remember the rules!), text book and JavaDocs when answering these questions!

Note: You **may not** use the drag-and-drop UI creation tools of IntelliJ, Netbeans, Jenuity or any other IDE for this section! If you do, you will be given **zero!!**

- 1) Prior to JavaFX, a *property* P of type T was defined by the presence of one or both of the methods named `getP` and `setP`. The signatures for these methods is as follows: (15)
- `void setP(T value)` – i.e. returns nothing, but accepts a single parameter of type T
 - `T getP()` – i.e. returns a value of type T , but accepts no parameters

The presence of the `setP` method indicates *write access* to the property P and the presence of the `getP` method indicates *read access* to the property P .

For example, if a class had the following methods:

```
Integer getWidth()           Class<?> getClass()
void setWidth(Integer value)

String getName()            int getCount()
void setName(String value)  float getArea(float x, float y)
```

then the class has four *properties*, namely:

- Width, type `Integer`, read and write access
- Name, type `String`, read and write access
- Class, type `Class<?>`, read only access
- Count, type `int`, read only access

Area is not a property since the `getArea` method has two parameters.

For this task, you are to write a method called `displayProperties`, that accepts a `Class` object as a parameter and, using reflection, determines the *properties* of the `Class`. Once these properties have been determined, the method must display the name, type and read/write access for each property. For simplicity, you may assume that:

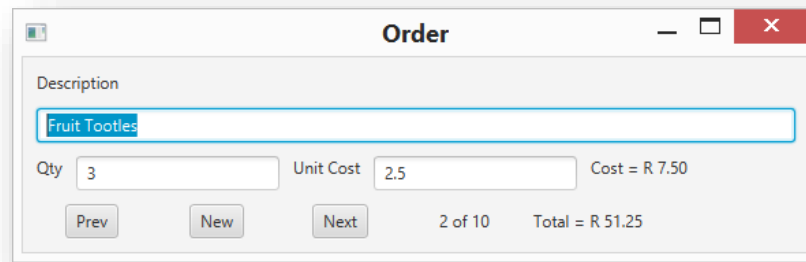
- private and protected properties may be ignored;
- if the type for the getter and setter do not match, simply specify ignore this fact.

The expected output for the `Stage` class is shown below (showing only the first few lines for brevity):

```
Properties for class Stage
r Owner (Window)
rw MinHeight (double)
r Icons (ObservableList)
w Maximized (void)
w Iconified (void)
w Resizable (void)
rw OnShown (EventHandler)
rw EventDispatcher (EventDispatcher)
:
```

Hint: you may make use of the `startsWith` and `substring` methods of the `String` class to assist you. If a method returns nothing, then its return type is `void.class`

- 2) For this task, you are to write a simple Java FX application that looks something like the screenshot shown below: (30)



The application is to allow a user to enter entries in an order. Each entry has a text description, quantity, unit cost and entry cost. The entry cost is calculated as the product of the quantity and unit cost. Each of these fields must be bound to a property in the model and changes should be reflected immediately throughout the UI. For example, changing either the quantity or unit cost of an entry should cause the entry's cost to be recalculated and displayed, which in turn should cause the order total to be recalculated and displayed.

The user is able to navigate through the entries using the previous and next buttons. A label to the right of the buttons displays the current entry's position in the entries collection.

On the bottom right the total of all the entries is displayed.

A new entry can be added by clicking on the new button. This entry is added to the end of the entries list.

Be sure to make use of JavaFX properties, binding and MVC in your solution.

The mark break down for this question is as follows:

- Creation of UI (5 marks)
- Model (4 marks)
- Controller (18 marks)
- Correct use of properties, bindings & MVC (3 marks)

- 3) Write a **console** application that: (10)
1. Connects to the addresses database found in the SQL Server. The server name is OPENBOX\WRR and the database name is "WRAP301ST2". The username is "ST2User" and the password is "May2015".
 2. Asks the user to enter the starting letter(s) of people's last name
 3. Uses the information entered in #2 to select all the people whose last name starts with the letters specified in #2
 4. Displays the information of the people obtained in #3
 5. Terminates the connection with the database

Once completed:

- Remember to put your **name and student number at the top of each .java file**. There is *always* someone who forgets!!
- **SAVE your work to the T:**
- Remember that if you do not save all your work to the T:, there is nothing to mark, i.e. expect 0!
- You may not leave the venue until the last 15 minutes of the test.